SJ

SmartJar

Technical Documentation — Full Stack Fintech Platform

Spring Boot 3 · React 18 + Vite · PostgreSQL · Razorpay · JWT + MPIN · 24K Digital Gold

Version 1.0 · April 2026 · Confidential

📋 SECTION 1

System Overview

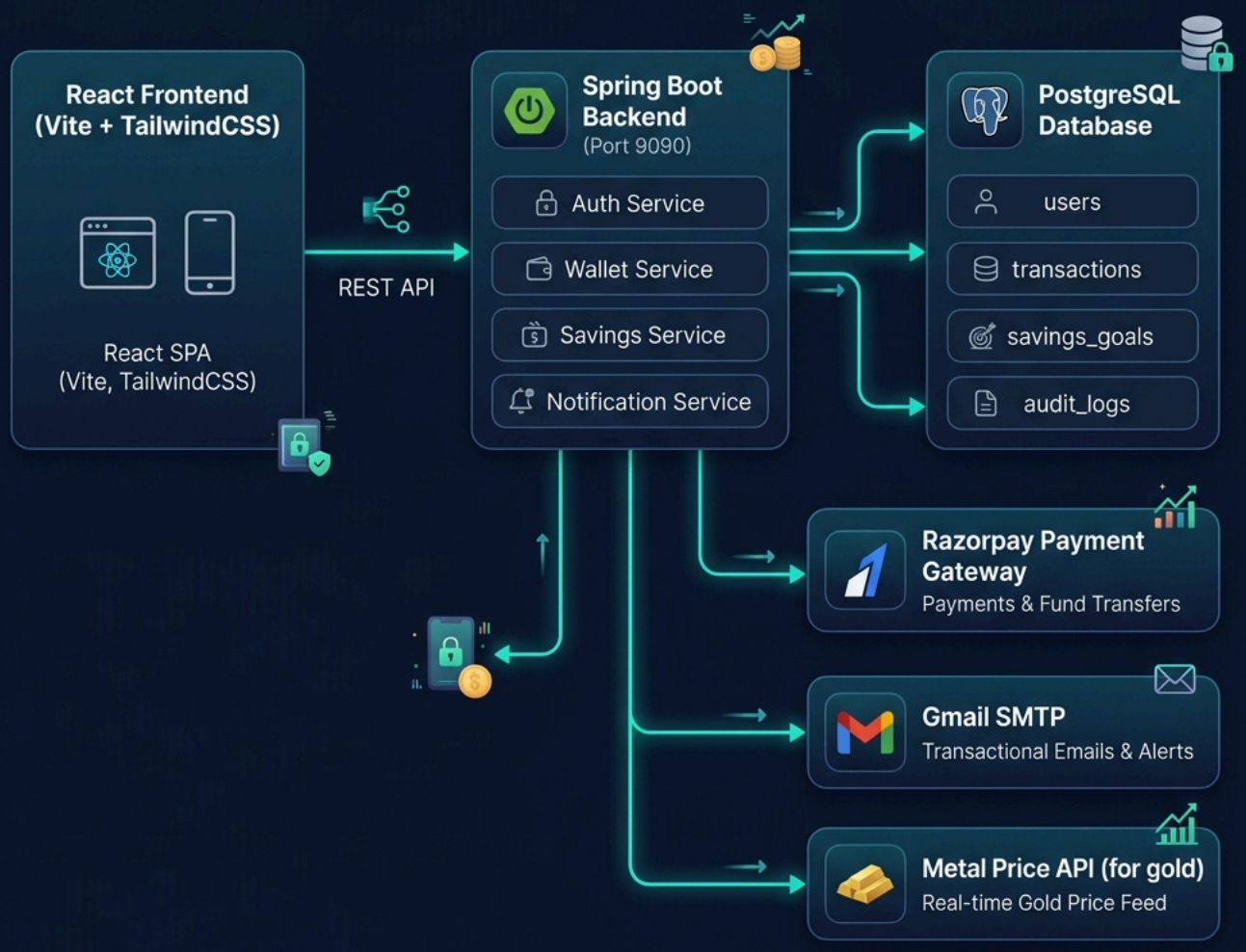
SmartJar is a production-grade full-stack fintech platform providing digital wallet management, UPI payments, auto-savings, and real-time 24K gold investment.

LAYER	TECHNOLOGY	DETAILS
Frontend	React 18 + Vite + TailwindCSS	11 pages: Login, Dashboard, Wallet, Gold, Savings, Bills, Settings, Send, QR...
Backend	Spring Boot 3 (Java 17)	REST API on port 9090, 11 controllers
Database	PostgreSQL 14+	7 tables, UUID PKs, DECIMAL precision
Auth	JWT + MPIN + Device Fingerprint	256-bit secret, 24h token expiry, OTP via email
Payments	Razorpay	Order creation + HMAC webhook verification
Email	Gmail SMTP	OTP delivery via Spring Mail / JavaMailSender
Gold API	Metal Price API	Live INR/XAU, 15-min cache, ₹15,329 fallback

Architecture Diagram

SmartJar

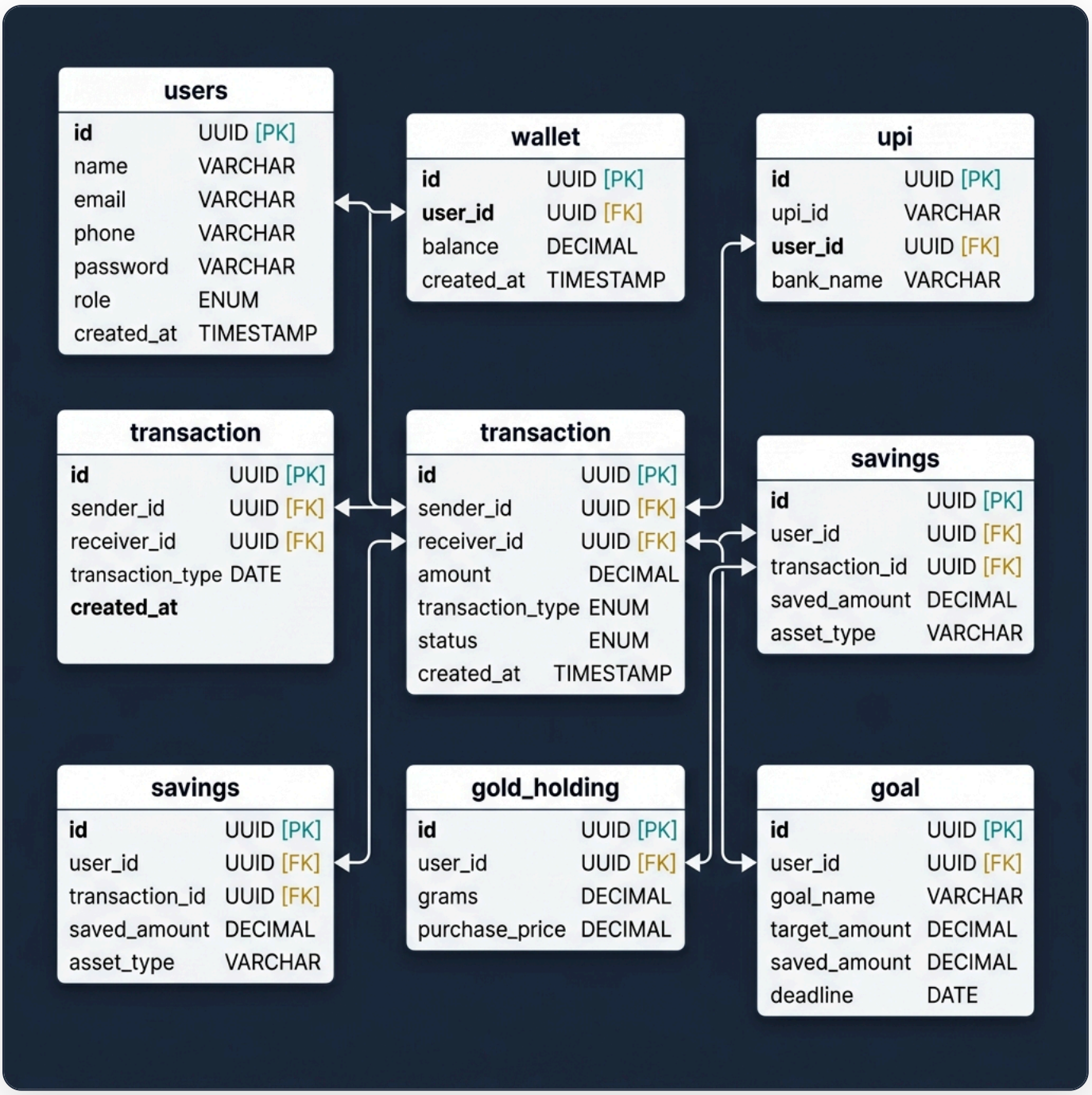
Digital Wallet & Savings Platform Architecture



SECTION 2

Database Schema

PostgreSQL with 7 normalised tables, all using UUID primary keys and timestamp audit columns.



DDL Script

```
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- Users (core identity table)
CREATE TABLE users (
    id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    name        VARCHAR(255) NOT NULL,
```

```sql
    email      VARCHAR(255) UNIQUE NOT NULL,
    phone      VARCHAR(20),
    password   VARCHAR(255) NOT NULL,   -- BCrypt hashed
    role       VARCHAR(50)  DEFAULT 'ROLE_USER',
    created_at TIMESTAMP    DEFAULT CURRENT_TIMESTAMP
);

-- Wallet (one per user)
CREATE TABLE wallet (
    id         UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id    UUID REFERENCES users(id),
    balance    DECIMAL(15,2) DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- UPI IDs linked to users
CREATE TABLE upi (
    id        UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    upi_id    VARCHAR(255) UNIQUE NOT NULL,
    user_id   UUID REFERENCES users(id),
    bank_name VARCHAR(100)
);

-- Immutable transaction ledger
CREATE TABLE transaction (
    id               UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    sender_id        UUID REFERENCES users(id),
    receiver_id      UUID REFERENCES users(id),
    amount           DECIMAL(15,2) NOT NULL,
    transaction_type VARCHAR(50) NOT NULL,  -- SEND | RECEIVE | BILL | SAVINGS
    status           VARCHAR(50) NOT NULL,  -- SUCCESS | PENDING | FAILED
    created_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Auto-savings records (round-up / fixed / percentage)
CREATE TABLE savings (
    id             UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id        UUID REFERENCES users(id),
    transaction_id UUID REFERENCES transaction(id),
    saved_amount   DECIMAL(15,2) NOT NULL,
    asset_type     VARCHAR(50)   NOT NULL,  -- CASH | GOLD
    created_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Gold holdings: buy = +grams, sell = -grams (net aggregated)
CREATE TABLE gold_holding (
    id             UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id        UUID REFERENCES users(id),
    grams          DECIMAL(10,4) NOT NULL,
    purchase_price DECIMAL(15,2) NOT NULL,
```

```sql
    created_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- User-defined savings goals
CREATE TABLE goal (
    id            UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id       UUID REFERENCES users(id),
    goal_name     VARCHAR(255) NOT NULL,
    target_amount DECIMAL(15,2) NOT NULL,
    saved_amount  DECIMAL(15,2) DEFAULT 0,
    deadline      TIMESTAMP,
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

🔐 SECTION 3

Security Architecture

Three independent authentication layers protect every user action and sensitive transaction.

🔑 **Layer 1 — JWT Bearer Token**
Attached to every request via Authorization header. 256-bit HMAC secret. 24h expiry.

📱 **Layer 2 — Device Fingerprinting**
Browser fingerprint stored in localStorage. New/unrecognized device triggers OTP to email.

🔢 **Layer 3 — MPIN Authorization**
Required for bill payments and P2P transfers. 6-digit PIN enforced server-side.

Device Fingerprint — Login.jsx

```
// Generate & persist device ID on first browser visit
useEffect(() => {
  let dId = localStorage.getItem('smartjar_device_id');
  if (!dId) {
    dId = 'browser_' + Math.random().toString(36).substring(2, 11);
    localStorage.setItem('smartjar_device_id', dId);
  }
  setDeviceId(dId);
}, []);

// Login request includes deviceId; backend compares against known devices
const res = await api.post('/auth/login', { email, password, deviceId });
if (res.data.requiresDeviceVerification) {
  setShowOtp(true); // OTP form shown for unknown device
}
```

JWT Configuration

```
# application.properties
app.jwt.secret=${JWT_SECRET:MyVeryLongAndSecureJwtSecretKeyForJarAppThatIsAtLeast256BitsLon
g!}
```

```
app.jwt.expiration-ms=86400000          # 24 hours
app.jwt.refresh-expiration-ms=604800000 # 7 days
```

SECTION 4

Configuration Reference

```
# application.properties
spring.application.name=smartjar-backend
server.port=9090

# PostgreSQL + HikariCP pool
spring.datasource.url=jdbc:postgresql://localhost:5432/smartjar
spring.datasource.username=jaruser
spring.datasource.password=1234
spring.datasource.hikari.maximum-pool-size=10
spring.datasource.hikari.connection-timeout=30000

# Razorpay (inject via env vars in production)
razorpay.key.id=${RAZORPAY_KEY_ID:rzp_test_ ... }
razorpay.key.secret=${RAZORPAY_KEY_SECRET: ... }
razorpay.webhook.secret=${RAZORPAY_WEBHOOK_SECRET: ... }

# Gmail SMTP for OTP delivery
spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=${MAIL_USER:your@gmail.com}
spring.mail.password=${MAIL_PASS:app-password}
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true

# Gold API — Metal Price API
gold.api.url=${GOLD_API_URL:https://api.metalpriceapi.com/v1/latest}
gold.api.key=${GOLD_API_KEY:your_key}
app.gold.cache-ttl-minutes=15

# Logging / Actuator
logging.level.com.smartjar=INFO
management.endpoints.web.exposure.include=health
```

SECTION 5

REST API Reference

Base URL: `http://localhost:9090/api` | CORS: `localhost:5173`, `localhost:3000`

Auth Endpoints — /api/auth

POST **/api/auth/register**

Register new user — wallet auto-created

```
// Request
{ "name": "Vinay Kumar", "email": "vinay@example.com",
  "password": "SecurePass@123", "phone": "9876543210" }

// Response
{ "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 ... ",
  "user": { "id": "3fa85f64- ... ", "name": "Vinay Kumar",
            "email": "vinay@example.com" },
  "requiresDeviceVerification": false }
```

POST **/api/auth/login**

Login with email + password + device fingerprint

```
// Request
{ "email": "vinay@example.com", "password": "SecurePass@123",
  "deviceId": "browser_k9×2m3qr7" }

// Response — Known Device
{ "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 ... ",
  "user": { ... }, "requiresDeviceVerification": false }

// Response — New Device (triggers email OTP)
{ "requiresDeviceVerification": true, "message": "OTP sent to email" }
```

POST **/api/auth/verify-device?email=&deviceId=&otp=**

Verify new device with emailed 6-digit OTP

```
// Response
{ "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 ... ",
  "user": { "id": " ... ", "name": "Vinay Kumar", "email": " ... " } }
```

Wallet Endpoints — /api/wallet

GET `/api/wallet/{userId}`

Fetch live wallet balance

```
// Response
{ "balance": 12500.00, "currency": "INR" }
```

POST `/api/wallet/{userId}/add?amount=500.00`

Credit wallet after Razorpay payment

```
// Response
{ "balance": 13000.00, "currency": "INR" }
```

Gold Endpoints — /api/gold

GET `/api/gold/price`

Live gold price in INR/gram from Metal Price API

```
// Response
{ "pricePerGram": 8734.56, "currency": "INR" }

// Live fetch logic (Troy ounce → grams conversion)
double pricePerGram = (inr / xau) / 31.1034768;
return BigDecimal.valueOf(pricePerGram).setScale(2, RoundingMode.HALF_UP);
```

POST `/api/gold/buy`

Purchase digital gold — deducted from wallet

```
// Request
{ "userId": "3fa85f64-...", "amount": 5000.00 }

// Response
{ "status": "SUCCESS", "goldBoughtGrams": 0.5726,
  "newWalletBalance": 7500.00 }
```

POST `/api/gold/sell`

Sell gold at live price — credited to wallet

```
// Request
{ "userId": "3fa85f64-...", "grams": 0.25 }
```

```
// Response
{ "status": "SUCCESS", "goldSoldGrams": 0.25,
  "cashReceived": 2183.64, "newWalletBalance": 9683.64 }
```

GET **/api/gold/portfolio/{userId}**

Total gold holdings and current market value

```
// Response
{ "totalGrams": 1.3274, "currentValue": 11593.45 }
```

Savings Endpoints — /api/savings

GET **/api/savings/total/{userId}**

Cumulative auto-savings amount

```
// Response
{ "total": 3245.75 }
```

POST **/api/savings/prefs/{userId}**

Update autopilot savings mode

```
// Request — enable round-up mode
{ "roundUp": true, "fixedAmount": false, "percentage": false }

// Response
{ "status": "SUCCESS" }
```

Payment Endpoints — /api/payment

POST **/api/payment/create-order**

Create Razorpay order for wallet top-up

```
// Request
{ "amount": 50000, "currency": "INR", "receipt": "rcpt_001" }

// Response
{ "orderId": "order_PnF9abcXYZ", "amount": 50000, "currency": "INR" }
```

POST

/api/payment/verify

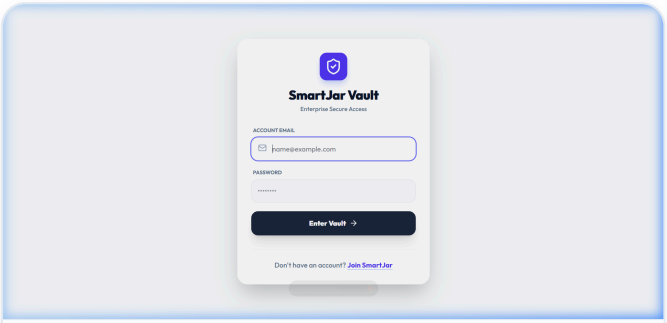
Verify Razorpay HMAC signature and credit wallet

```
// Request
{ "razorpay_order_id": "order_PnF9abcXYZ",
  "razorpay_payment_id": "pay_PnFabc123",
  "razorpay_signature": "hmac_sha256_signature",
  "userId": "3fa85f64-…", "amount": 500.00 }
```

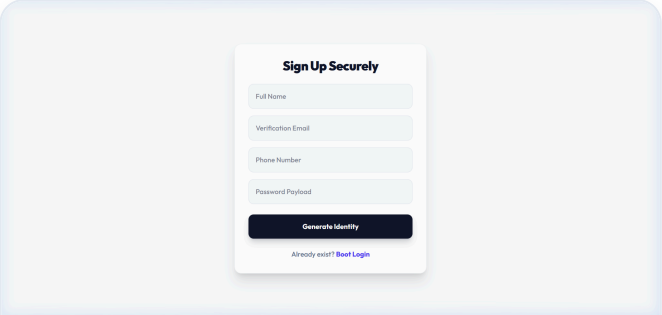
SECTION 6

UI Screenshots

All pages captured from the live Vite dev server at localhost:5174.

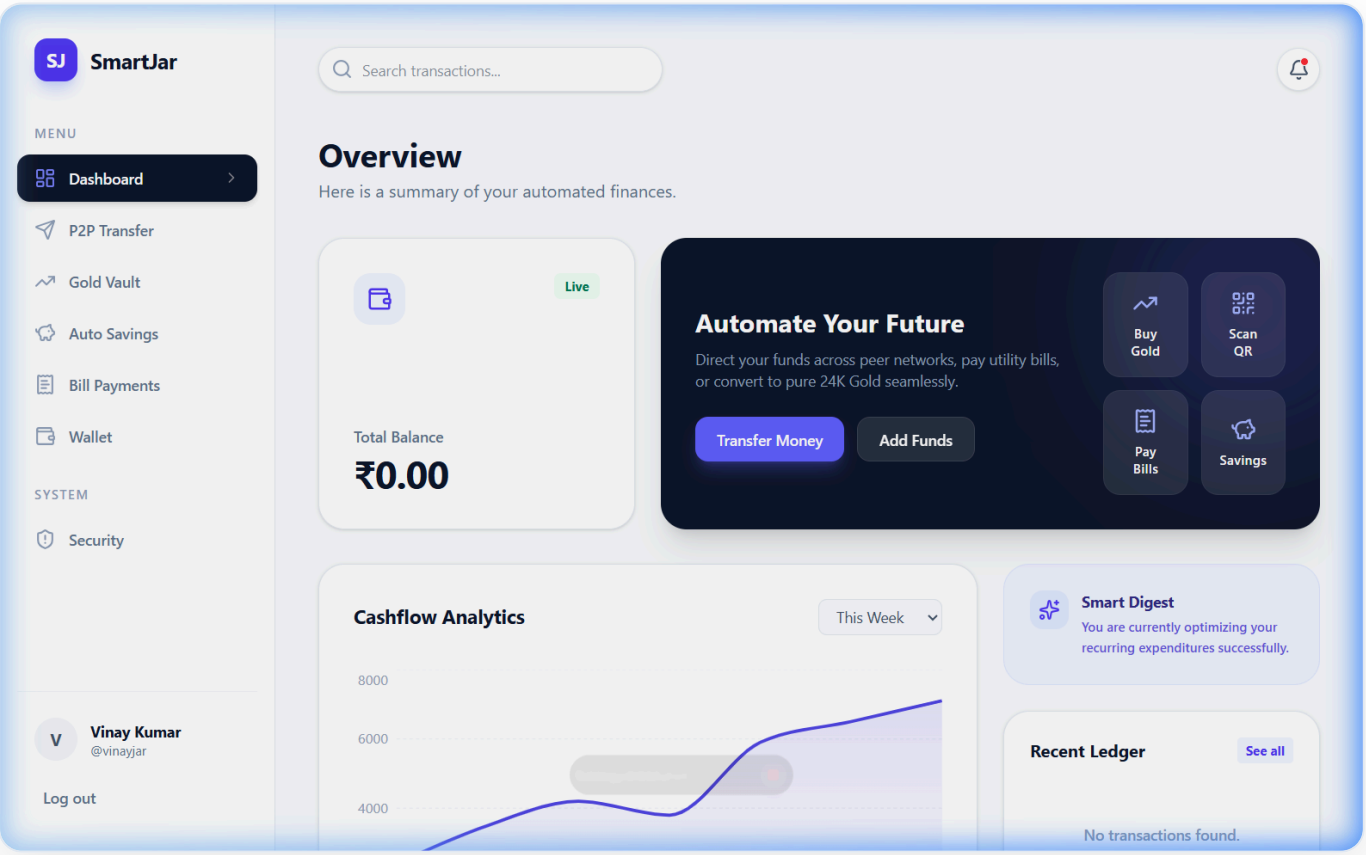


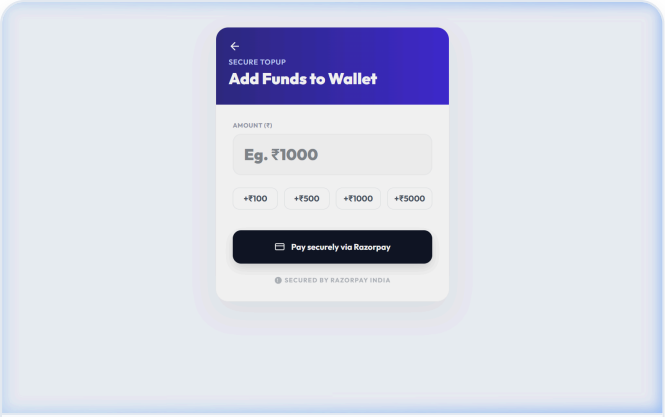
Login — SmartJar Vault / login



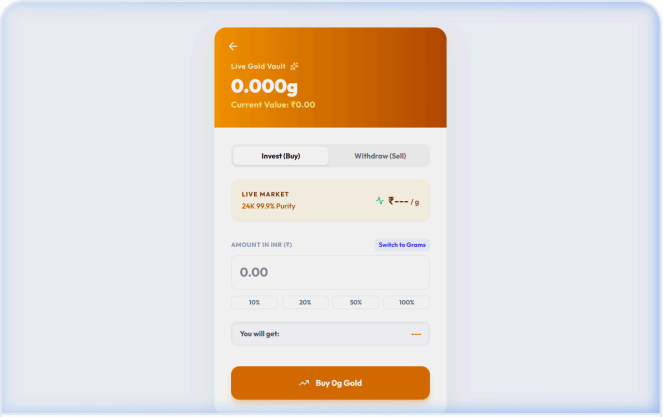
Register — Sign Up Securely / register

Dashboard — Financial Overview

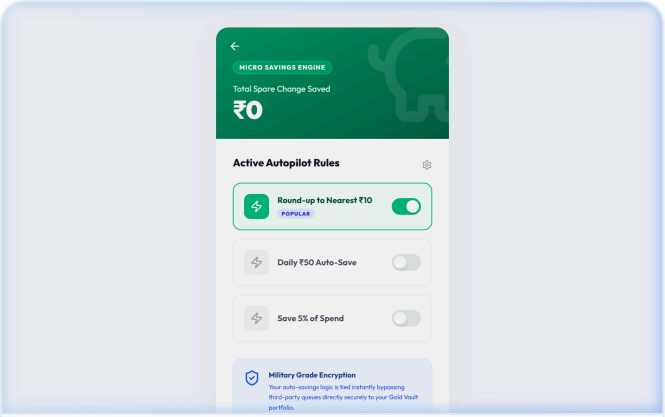




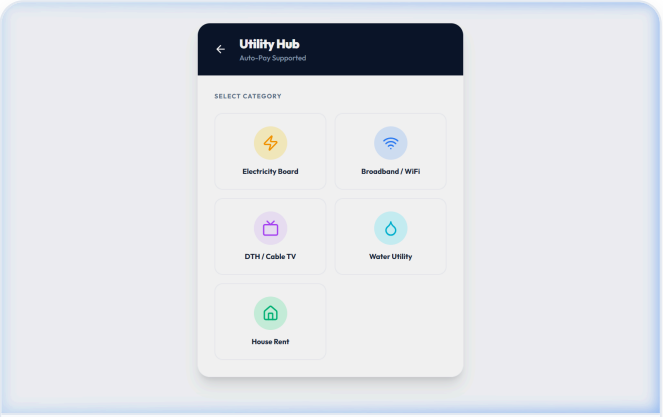
Wallet — Add Funds / wallet



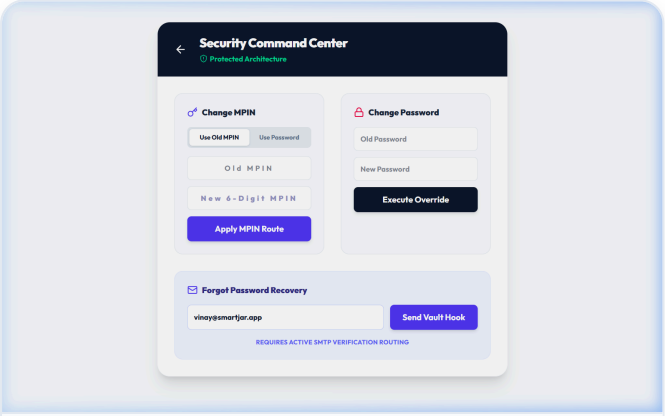
Gold Vault — Buy & Sell / gold



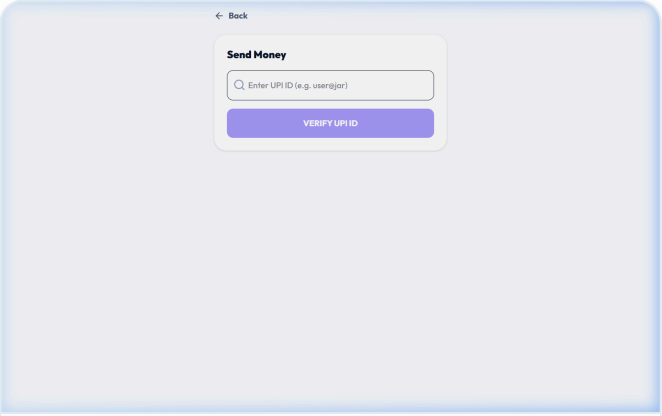
Micro Savings Engine / savings



Utility Hub — Bills / bills

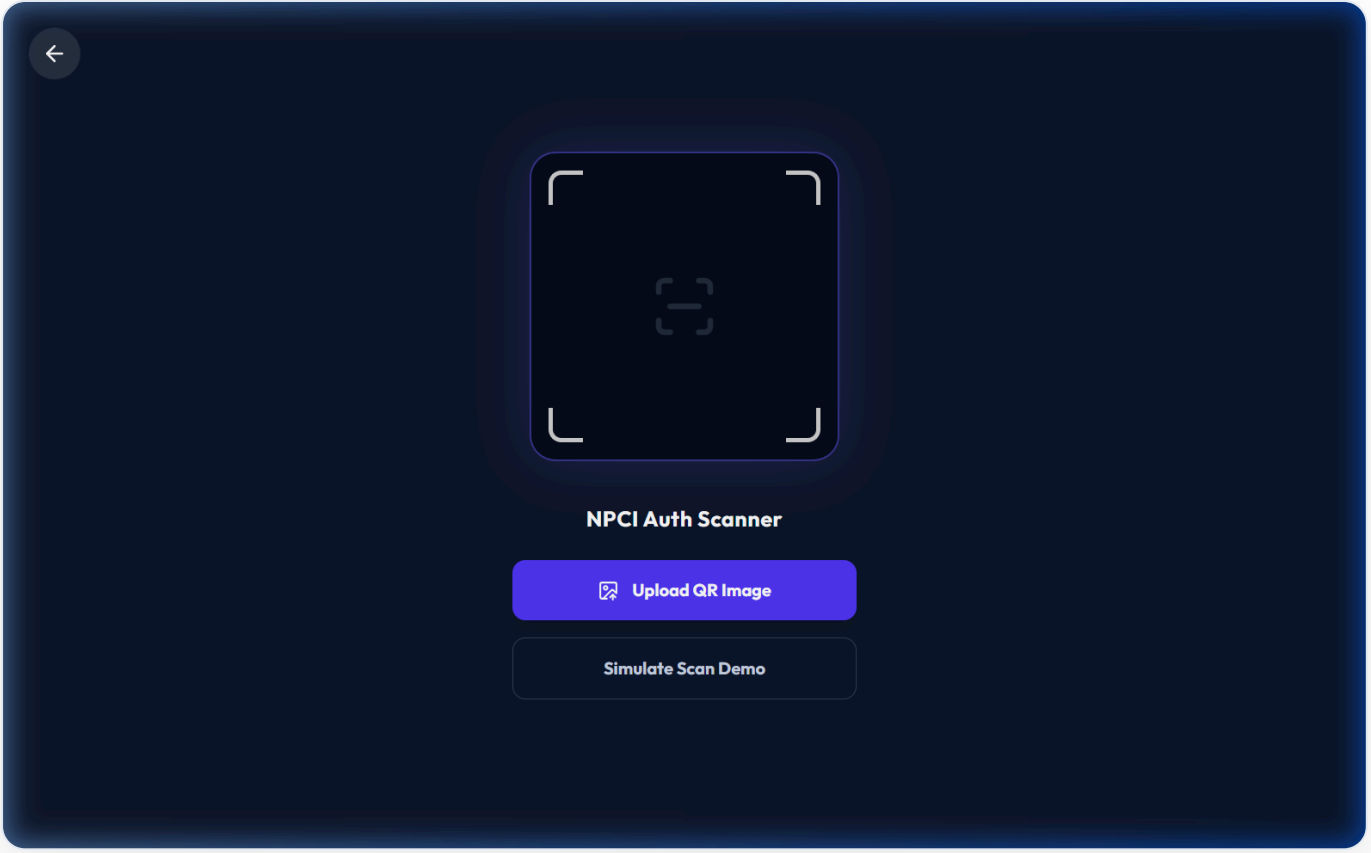


Security Command Center / settings



Send Money — UPI P2P / send

QR Scanner — NPCI Auth Scanner


NPCI Auth Scanner
Upload QR Image
Simulate Scan Demo

🚀 SECTION 7

Setup & Deployment

Prerequisites

REQUIREMENT	MINIMUM VERSION
Java JDK	17+
Apache Maven	3.9+
Node.js	18+
PostgreSQL	14+

1. Database Setup

```
-- As PostgreSQL superuser
CREATE USER jaruser WITH PASSWORD '1234';
CREATE DATABASE smartjar OWNER jaruser;
\c smartjar
\i schema.sql
```

2. Run Backend

```
# Windows
cd smartjar-backend
mvnw.cmd spring-boot:run

# macOS / Linux
./mvnw spring-boot:run

# Backend starts at http://localhost:9090
```

3. Run Frontend

```
cd smartjar-backend/smartjar-frontend
npm install
npm run dev
```

```
# Frontend starts at http://localhost:5173
```

4. Production Environment Variables

```
export JWT_SECRET="your-256-bit-secret-here"
export RAZORPAY_KEY_ID="rzp_live_ ... "
export RAZORPAY_KEY_SECRET=" ... "
export RAZORPAY_WEBHOOK_SECRET=" ... "
export MAIL_USER="your@gmail.com"
export MAIL_PASS="gmail-app-password"
export GOLD_API_KEY="your-metal-price-api-key"
```

5. Docker Deployment

```
# Dockerfile
FROM eclipse-temurin:17-jdk-alpine
WORKDIR /app
COPY target/*.jar app.jar
EXPOSE 9090
ENTRYPOINT ["java", "-jar", "app.jar"]

# Build & Run
./mvnw clean package -DskipTests
docker build -t smartjar-backend .
docker run -p 9090:9090 \
  -e JWT_SECRET="your-secret" \
  -e RAZORPAY_KEY_ID="rzp_ ... " \
  -e MAIL_USER="your@gmail.com" \
  smartjar-backend
```

Project Structure

```
smartjar-backend/
├── src/main/java/com/smartjar/
│   ├── controller/   AuthController · WalletController · GoldController
│   │                 SavingsController · BillsController · PaymentController
│   │                 UpiController · TransactionController · SecurityController
│   ├── entity/       User · Wallet · UPI · Transaction · Savings · GoldHolding · Goal
│   ├── repository/   Spring Data JPA repositories
│   ├── service/      AuthService · WalletService · EmailService · ...
│   ├── security/     JwtUtil · JwtFilter · MPINService
│   ├── dto/          RegisterRequest · AuthRequest · AuthResponse · WalletResponse
│   └── scheduler/    Auto-savings cron jobs
```

```
├── smartjar-frontend/src/pages/
│   └── Login · Register · Dashboard · Wallet · Gold · Savings
│       Bills · PayBills · SendMoney · QrScanner · Settings
├── schema.sql
├── application.properties
└── Dockerfile
```